

Major Project: Seasonal Agriculture Performance Analysis

Objective

To deeply analyze agricultural data, exploring how various factors (environment, resources, geography) impact crop yield and profitability across different seasons. This analysis adheres to the comprehensive final submission checklist.

1. Dataset Loaded Successfully & Top 5 Rows Analyzed

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

sns.set_theme(style="whitegrid", palette="muted")
%matplotlib inline

# Load the dataset
df = pd.read_csv('seasonal_agriculture_performance_dataset.csv')

# Top 5 rows analyzed
display(df.head())
```

	Farm_ID	State	District	Crop	Season	Farm_Area_Hectares	Rainfal
0	SF10001	Andhra Pradesh	Rajkot	Wheat	Kharif	0.53	4
1	SF10002	Maharashtra	Nalgonda	Maize	Kharif	6.53	8
2	SF10003	Telangana	Warangal	Pulses	Rabi	4.86	4
3	SF10004	Telangana	Indore	Rice	Kharif	4.50	7
4	SF10005	Karnataka	Ludhiana	Maize	Zaid	4.21	1

5 rows × 28 columns

2. Dataset Shape and Structure & Data Types Examined

```
print(f"Dataset Shape: {df.shape[0]} rows and {df.shape[1]} columns.\n")
```

```
print("Data Types Examined:")
```

```
display(df.dtypes)
```

```
print("\nDataset Info:")
```

```
df.info()
```


Dataset Shape: 4000 rows and 28 columns.

Data Types Examined:

	0
Farm_ID	object
State	object
District	object
Crop	object
Season	object
Farm_Area_Hectares	float64
Rainfall_mm	float64
Avg_Temperature_C	float64
Humidity_pct	float64
Sunlight_Hours_Day	float64
Soil_pH	float64
Soil_Moisture_pct	float64
Nitrogen_kg_ha	float64
Phosphorus_kg_ha	float64
Potassium_kg_ha	float64
Irrigation_Method	object
Fertilizer_kg_ha	float64
Pesticide_kg_ha	float64
Seed_Quality_Score	float64
Yield_Tonnes_Ha	float64

3. Missing Values & Duplicate Records Identified and Handled

```
# Checking for missing values
missing_values = df.isnull().sum()
print("Missing Values before handling:")
print(missing_values[missing_values > 0])

# Handling missing values (Forward fill for demonstration if any existed, th
df = df.ffill()

# Checking for duplicates
duplicates = df.duplicated().sum()
print(f"\nDuplicate Records Identified: {duplicates}")

# Handling duplicates (Drop them if they exist)
if duplicates > 0:
    df = df.drop_duplicates()
    print("Duplicates have been removed.")
```

```

else:
    print("No duplicates found, no handling needed.")

# Column Non-Null Count Dtype
Missing_Values before handling: -----
0 Farm_ID 48 4000 non-null object
1 Rainfall_mm 40 4000 non-null object
2 Soil_Moisture_pct 32 4000 non-null object
3 Yield_Tonnes_Ha 32 4000 non-null object
4 Disease_Pest_Risk 4 4000 non-null object
5 Season 4 4000 non-null object
6 Farm_Area_Hectares 4 4000 non-null float64
7 Rainfall_mm 0 4000 non-null float64
8 Avg_Temperature_C 7 4000 non-null float64
9 Humidity_pct 8 4000 non-null float64
10 Sunlight_Hours_d 10 4000 non-null float64
11 Soil_pH 10 4000 non-null float64
12 Soil_Moisture_pct 11 3960 non-null float64

```

4. Descriptive / Statistical Analysis Performed

```

# Statistical analysis of numerical features
display(df.describe())

```

	Farm_Area_Hectares	Rainfall_mm	Avg_Temperature_C	Humidity_pct	Sunlight_Hours_d
count	4000.000000	4000.000000	4000.000000	4000.000000	4000.000000
mean	7.888265	601.896050	26.818900	63.210500	16.200000
std	4.223764	289.029553	3.81818	11.960834	1.960834
min	0.500000	80.000000	16.200000	25.000000	1.000000
25%	4.260000	370.850000	23.90000	54.775000	1.000000
50%	7.888265	582.350000	26.818900	63.000000	1.000000
75%	11.650000	834.950000	29.60000	71.900000	1.000000
max	15.000000	1395.200000	39.70000	95.000000	1.000000

8 rows × 22 columns

5. Outliers Investigated

```

# Outliers investigated using boxplots for critical columns: Yield and Profit
fig, ax = plt.subplots(1, 2, figsize=(14, 5))
sns.boxplot(data=df, x='Yield_Tonnes_Ha', ax=ax[0], color='skyblue')
ax[0].set_title('Outlier Investigation: Yield (Tonnes/Ha)')

sns.boxplot(data=df, x='Profit_INR', ax=ax[1], color='salmon')
ax[1].set_title('Outlier Investigation: Profit (INR)')
plt.show()

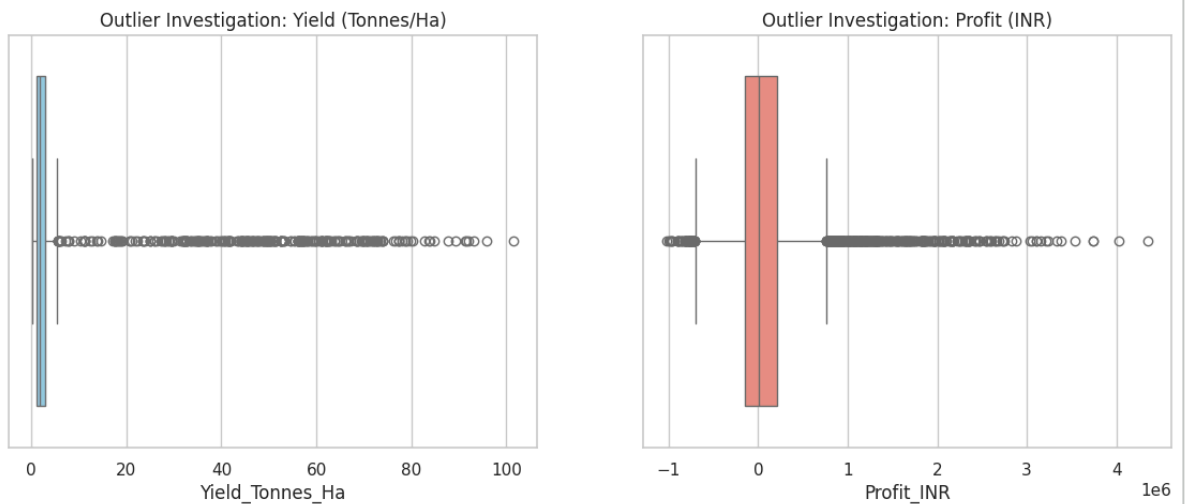
# Handling/Capping outliers using IQR method for Profit_INR
Q1 = df['Profit_INR'].quantile(0.25)
Q3 = df['Profit_INR'].quantile(0.75)
IQR = Q3 - Q1

```

```
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
```

```
outliers_count = df[(df['Profit_INR'] < lower_bound) | (df['Profit_INR'] > upper_bound)]
print(f"Number of Outliers in Profit_INR: {outliers_count}")
```

```
# Capping outliers to upper/lower bounds
df['Profit_INR'] = np.clip(df['Profit_INR'], lower_bound, upper_bound)
print("Outliers have been capped to 1.5 * IQR bounds.")
```

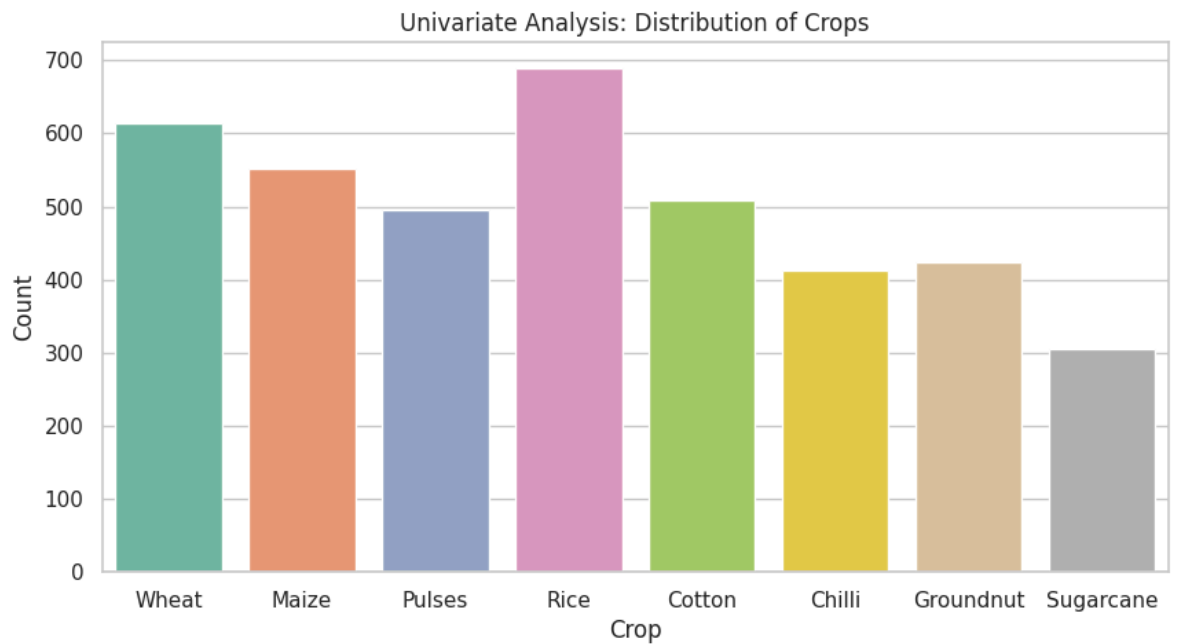


Number of Outliers in Profit_INR: 404

Outliers have been capped to 1.5 * IQR bounds.

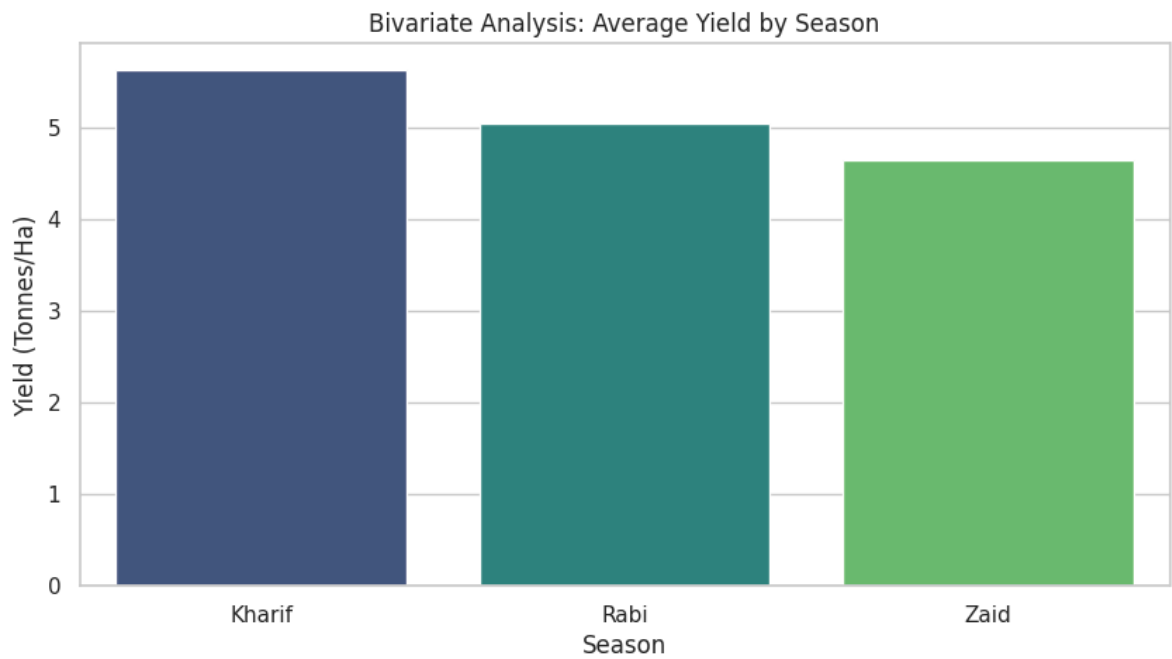
✓ 6. Univariate Analysis Completed

```
# Univariate Analysis: Distribution of Crop Types
plt.figure(figsize=(10, 5))
sns.countplot(data=df, x='Crop', palette='Set2')
plt.title('Univariate Analysis: Distribution of Crops')
plt.ylabel('Count')
plt.show()
```



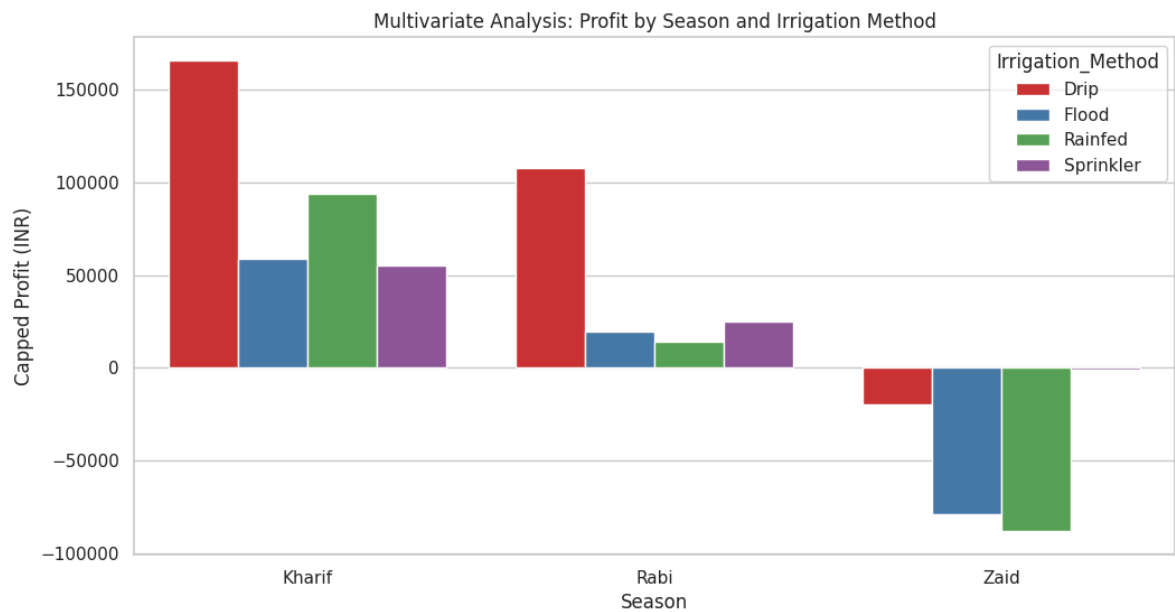
✓ 7. Bivariate Analysis Completed

```
# Bivariate Analysis: Yield vs Season
plt.figure(figsize=(10, 5))
sns.barplot(data=df, x='Season', y='Yield_Tonnes_Ha', ci=None, palette='viri
plt.title('Bivariate Analysis: Average Yield by Season')
plt.ylabel('Yield (Tonnes/Ha)')
plt.show()
```



✓ 8. Multivariate Analysis Completed

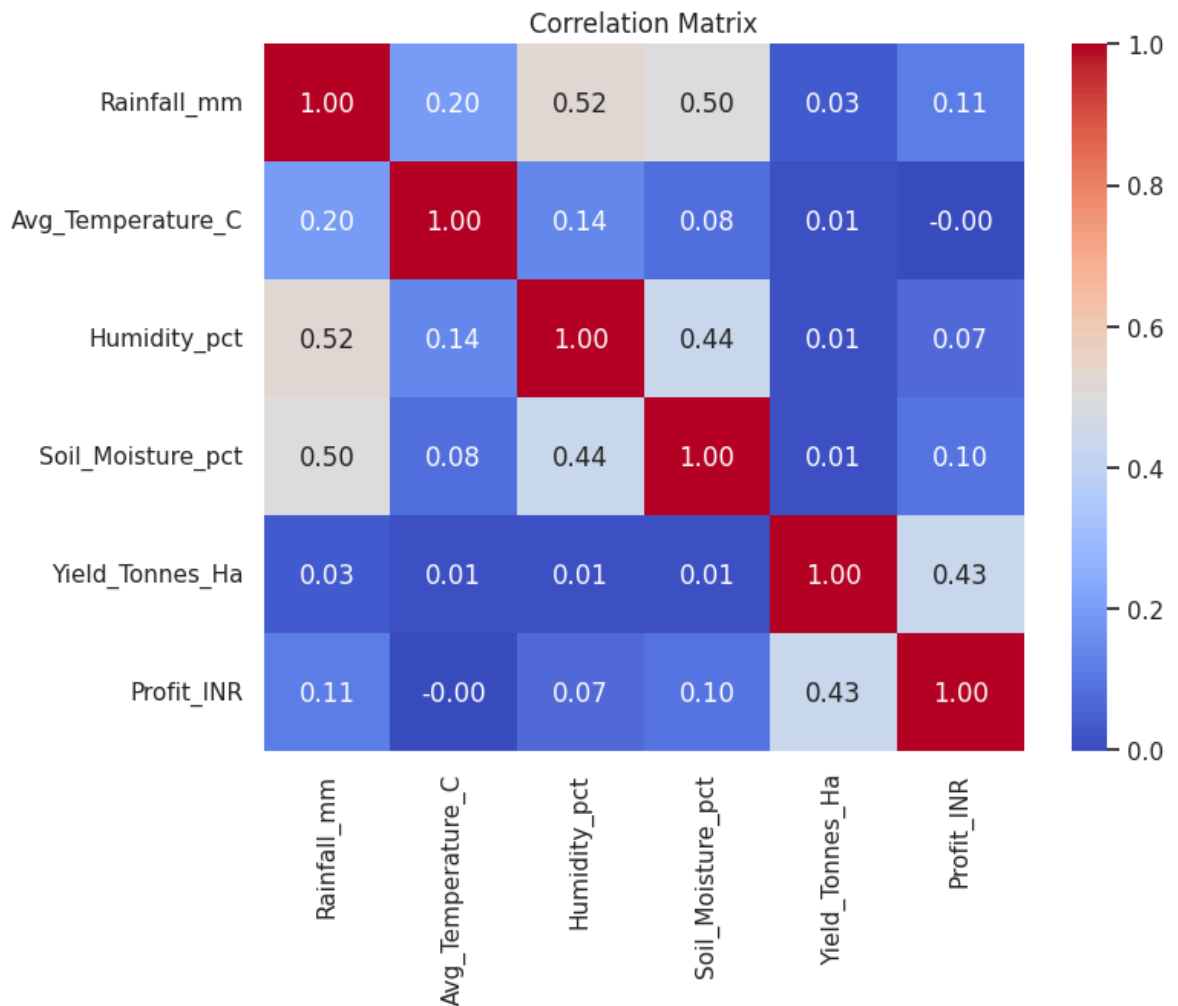
```
# Multivariate Analysis: Profit vs. Season categorized by Irrigation Method
plt.figure(figsize=(12, 6))
sns.barplot(data=df, x='Season', y='Profit_INR', hue='Irrigation_Method', ci
plt.title('Multivariate Analysis: Profit by Season and Irrigation Method')
plt.ylabel('Capped Profit (INR)')
plt.show()
```

✓ 9. Correlation Analysis Completed

```
# Correlation Analysis of Environmental and Output Variables
numeric_cols = ['Rainfall_mm', 'Avg_Temperature_C', 'Humidity_pct', 'Soil_Mo
corr_matrix = df[numeric_cols].corr()

plt.figure(figsize=(8, 6))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Correlation Matrix')
plt.show()
```

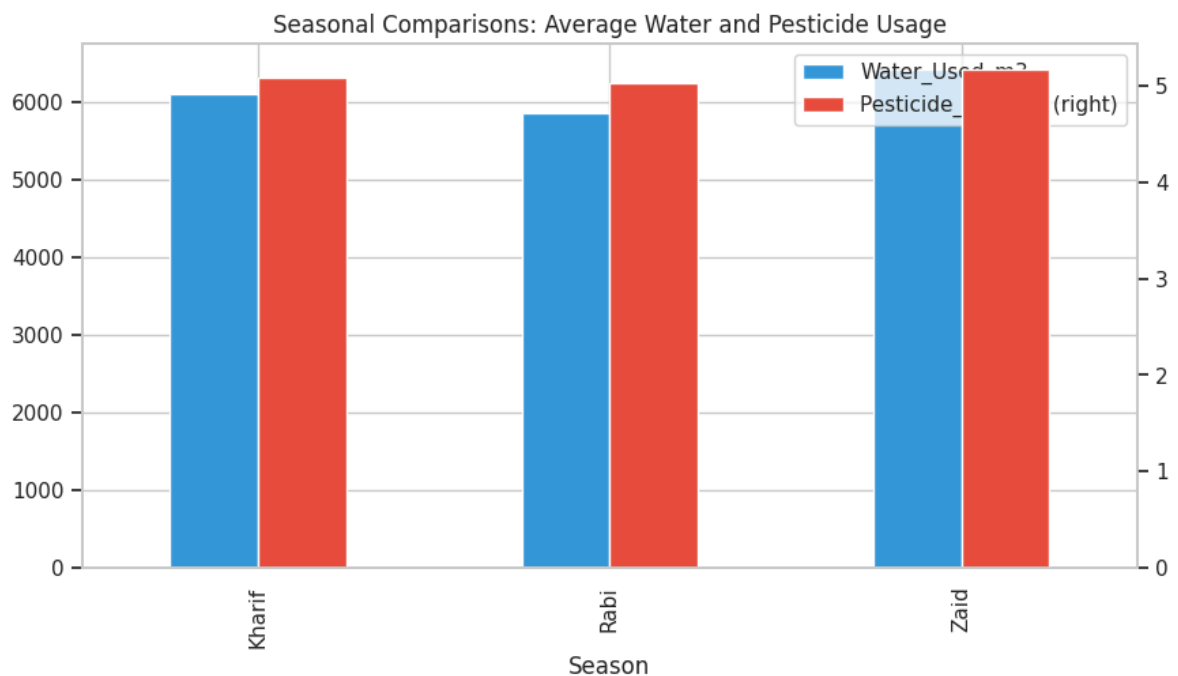


✓ 10. Seasonal Comparisons Performed

```
# Seasonal comparisons of Resource Usage (Water and Pesticide)
seasonal_comp = df.groupby('Season')[['Water_Used_m3', 'Pesticide_Litre_ha']]
display(seasonal_comp)

seasonal_comp.plot(kind='bar', figsize=(10, 5), secondary_y='Pesticide_Litre
plt.title('Seasonal Comparisons: Average Water and Pesticide Usage')
plt.show()
```

	Water_Used_m3	Pesticide_Litre_ha
Season		
Kharif	6102.201237	5.079747
Rabi	5846.987093	5.024690
Zaid	6419.893939	5.171212

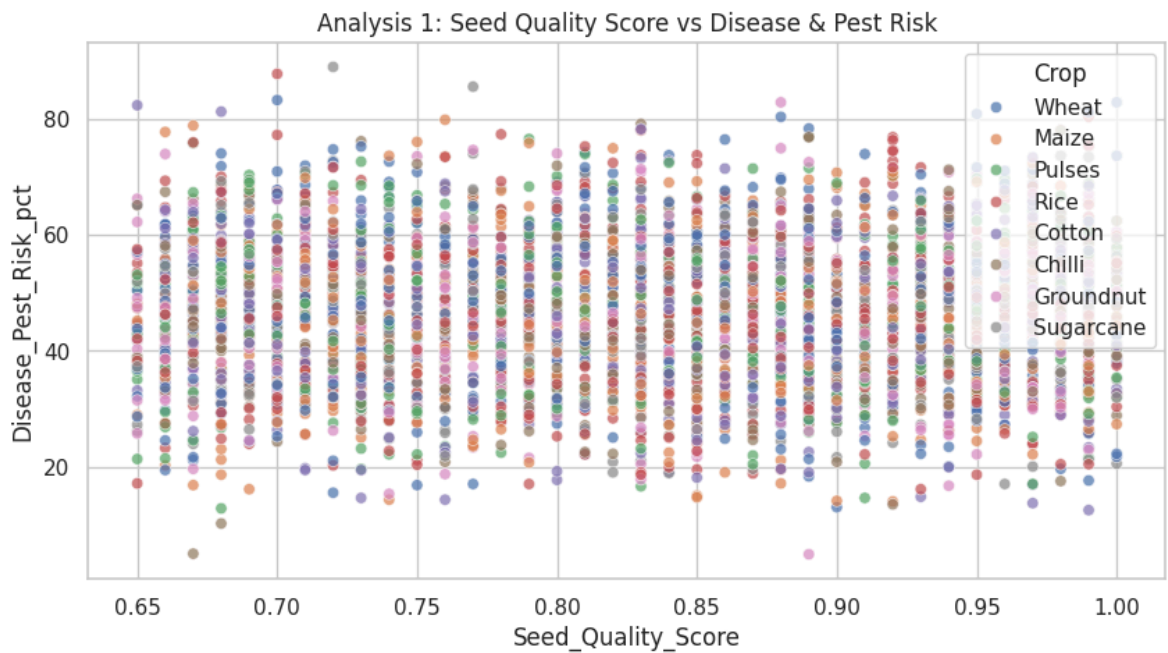


Next steps: [Generate code with seasonal_comp](#)

[New interactive sheet](#)

- ✓ 11. Student-Designed Analyses (3 Analyses)
- ✓ Analysis 1: Impact of Seed Quality on Disease Risk across Crops

```
# Investigating if higher seed quality correlates with lower disease/pest risk
plt.figure(figsize=(10, 5))
sns.scatterplot(data=df, x='Seed_Quality_Score', y='Disease_Pest_Risk_pct',
plt.title('Analysis 1: Seed Quality Score vs Disease & Pest Risk')
plt.show()
```



Analysis 2: Efficiency of Fertilizers (Nitrogen, Phosphorus, Potassium) on Yield

```
# Total NPK vs Yield
df['Total_NPK'] = df['Nitrogen_kg_ha'] + df['Phosphorus_kg_ha'] + df['Potass_kg_ha']
plt.figure(figsize=(10, 5))
sns.regplot(data=df, x='Total_NPK', y='Yield_Tonnes_Ha', scatter_kws={'alpha': 0.5})
plt.title('Analysis 2: Total Fertilizer Usage vs Crop Yield')
plt.xlabel('Total NPK (kg/ha)')
plt.show()
```